

Публикация и развёртывание приложения

Введение

На предыдущих лекциях было рассмотрено создание ASP.NET Web API: архитектура приложения, работа с базой данных, обработка файлов и обеспечение безопасности. Все эти темы изучались в контексте локальной разработки — приложение запускалось на рабочей машине и было доступно только в рамках локальной сети. В данной лекции рассматривается следующий шаг: как подготовить приложение к эксплуатации и сделать его доступным для реальных пользователей.

В этой лекции будут рассмотрены сборка приложения в режиме Release, концепция обратного прокси-сервера, последовательность действий по развёртыванию на удалённом Linux-сервере, а также обзор подхода с использованием Docker.

1. Сборка приложения для публикации

Режимы Debug и Release

В процессе разработки приложение запускается в режиме **Debug**. Этот режим оптимизирован для удобства разработчика: компилятор сохраняет отладочные символы, отключает часть оптимизаций и выводит подробную диагностическую информацию. В таком режиме приложение работает медленнее, потребляет больше памяти и может раскрывать внутренние детали реализации в сообщениях об ошибках.

Режим **Release** предназначен для эксплуатации. Компилятор применяет оптимизации, удаляет отладочные символы и отключает вывод диагностической информации. Уровень логирования по умолчанию также меняется: в режиме Release не выводятся подробные логи уровня `Debug` и `Trace`, которые полезны при разработке, но избыточны и потенциально небезопасны в продакшене — они могут раскрывать структуру запросов, параметры и внутренние состояния приложения.

Помимо этого, ASP.NET изменяет своё поведение в зависимости от переменной окружения `ASPNETCORE_ENVIRONMENT`. Наглядный пример — подключение Swagger. В процессе разработки Swagger необходим: он предоставляет удобный интерфейс для тестирования endpoint'ов. В продакшене же он должен быть отключён, поскольку раскрывает полную структуру API, все маршруты и модели данных, что является избыточной информацией для потенциального злоумышленника:

```
// Program.cs
// Подключение Swagger – только в режиме разработки
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
}
```

```
app.UseSwaggerUI();  
}
```

Благодаря проверке `app.Environment.IsDevelopment()` Swagger автоматически подключается только тогда, когда переменная `ASPNETCORE_ENVIRONMENT` имеет значение `Development`, и не подключается при значении `Production`.

Значение этой переменной устанавливается по-разному в зависимости от среды. При локальной разработке она задаётся в файле `launchSettings.json` и Visual Studio устанавливает её автоматически:

```
{  
  "profiles": {  
    "MyApp": {  
      "environmentVariables": {  
        "ASPNETCORE_ENVIRONMENT": "Development"  
      }  
    }  
  }  
}
```

На сервере переменная устанавливается на уровне операционной системы или в unit-файле `systemd`, что будет рассмотрено в разделе о развёртывании.

Команда `dotnet publish`

Для сборки приложения и подготовки его к развёртыванию используется команда `dotnet publish`. В отличие от `dotnet build`, которая просто компилирует код, `dotnet publish` формирует полный набор файлов, необходимых для запуска приложения на целевой машине.

Базовый вызов команды выглядит следующим образом:

```
dotnet publish -c Release -o ./publish
```

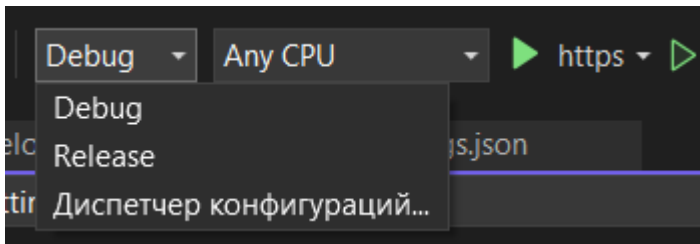
Параметр `-c Release` указывает режим сборки. Параметр `-o ./publish` задаёт папку, в которую будут помещены результирующие файлы. После выполнения команды папка `publish` будет содержать всё необходимое для запуска приложения.

При необходимости собрать приложение под конкретную платформу используется параметр `-r`:

```
# Сборка для Linux x64  
dotnet publish -c Release -o ./publish -r linux-x64
```

```
# Сборка для Windows x64
dotnet publish -c Release -o ./publish -r win-x64
```

Так же это можно сделать, через интерфейс Visual Studio, сделав сборку проекта с соответствующей конфигурацией.



Self-contained и framework-dependent

При публикации необходимо выбрать один из двух режимов, определяющих зависимость приложения от установленной на сервере версии .NET.

Framework-dependent — режим по умолчанию. Приложение публикуется без среды выполнения .NET и требует её предварительной установки на сервере.

Результирующие файлы занимают значительно меньше места — как правило, несколько мегабайт. Это предпочтительный выбор, когда окружение сервера находится под контролем и версия .NET может быть установлена заранее:

```
dotnet publish -c Release -o ./publish
```

Self-contained — приложение публикуется вместе со средой выполнения .NET. На сервере не требуется ничего устанавливать — достаточно скопировать папку и запустить исполняемый файл. Недосток — значительно больший размер публикации, порядка 60–100 МБ. Этот режим удобен, когда нет возможности управлять окружением сервера или требуется гарантированная версия среды выполнения:

```
dotnet publish -c Release -o ./publish -r linux-x64 --self-contained true
```

Содержимое папки publish

После выполнения `dotnet publish` папка публикации содержит следующие файлы:

```
publish/
├─ MyApp.dll          # скомпилированный код приложения
├─ MyApp.exe          # исполняемый файл (только для Windows или self-
contained)
├─ MyApp.runtimeconfig.json # конфигурация среды выполнения
├─ appsettings.json   # конфигурация приложения
└─ appsettings.Production.json
```

```
|— wwwroot/          # статические файлы
|— ...              # зависимости и библиотеки
```

Для запуска framework-dependent приложения на сервере используется команда:

```
dotnet MyApp.dll
```

Следует обратить внимание на то, что файл `appsettings.json` попадает в папку публикации. Как было рассмотрено в лекции по безопасности, этот файл не должен содержать секретных данных — они должны передаваться через переменные окружения на целевом сервере.

2. Обратный прокси-сервер

Kestrel и его ограничения

ASP.NET Web API запускается на встроенном веб-сервере Kestrel. Kestrel — это кроссплатформенный HTTP-сервер, разработанный Microsoft специально для ASP.NET. Именно он принимает входящие HTTP-запросы и передаёт их в конвейер обработки приложения.

Kestrel вполне способен принимать запросы напрямую из интернета, однако в продакшен-окружении это не рекомендуется по ряду причин. Kestrel не предназначен для работы с задачами, которые традиционно решаются на уровне веб-сервера: он не умеет эффективно раздавать статические файлы, не поддерживает гибкую настройку HTTPS, не обеспечивает балансировку нагрузки и предоставляет ограниченные возможности для защиты от атак и настройки логирования на уровне инфраструктуры. Кроме того, напрямую открывать порт приложения наружу означает отсутствие какого-либо буфера между интернетом и самим процессом .NET.

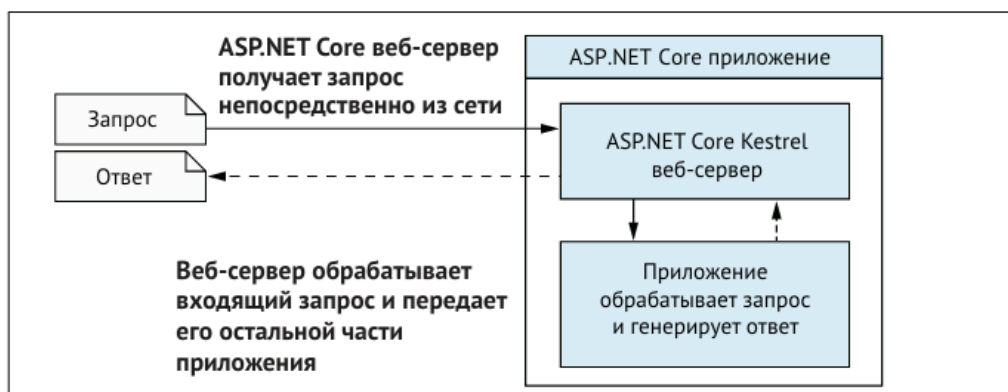
Для решения этих задач перед Kestrel размещается обратный прокси-сервер.

Что такое обратный прокси-сервер

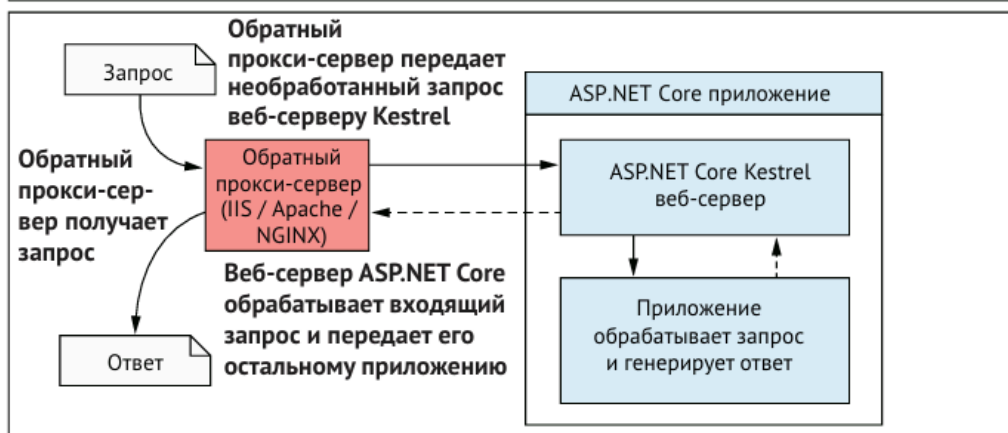
Обратный прокси — это сервер, который принимает все входящие запросы от клиентов и перенаправляет их к одному или нескольким серверам приложений. Клиент взаимодействует исключительно с обратным прокси и ничего не знает о том, что находится за ним.

Схема взаимодействия выглядит следующим образом:

ASP.NET Core приложения могут напрямую подключаться к сети для обработки запросов



ASP.NET Core приложения выполняются независимо от IIS и других прокси-серверов



Обратный прокси берёт на себя следующие задачи:

Терминация TLS. TLS (Transport Layer Security) — это криптографический протокол, обеспечивающий шифрование трафика между клиентом и сервером. Именно он лежит в основе HTTPS. Терминация TLS означает, что зашифрованное соединение завершается на уровне обратного прокси, а не доходит до самого приложения. Прокси принимает зашифрованный HTTPS-запрос от клиента, расшифровывает его и передаёт Kestrel уже в виде обычного HTTP внутри защищённой сети сервера.

Отдача статических файлов. Прокси сервер может значительно эффективнее Kestrel справляться с отдачей статики — изображений, стилей, скриптов. Запросы к статическим файлам обрабатываются прокси напрямую, не доходя до приложения.

Балансировка нагрузки. Обратный прокси может распределять входящие запросы между несколькими экземплярами приложения, что позволяет масштабировать систему горизонтально.

Защита и ограничение запросов. На уровне прокси можно ограничить размер тела запроса, настроить таймауты, блокировать подозрительные запросы и скрыть внутреннюю структуру инфраструктуры от клиентов.

Логирование на уровне инфраструктуры. Прокси фиксирует все входящие запросы независимо от того, дошли ли они до приложения.

Популярные решения

Nginx — наиболее распространённый выбор для Linux-серверов. Отличается высокой производительностью, минимальным потреблением ресурсов и гибкой конфигурацией. Именно Nginx будет использоваться в практической части данной лекции.

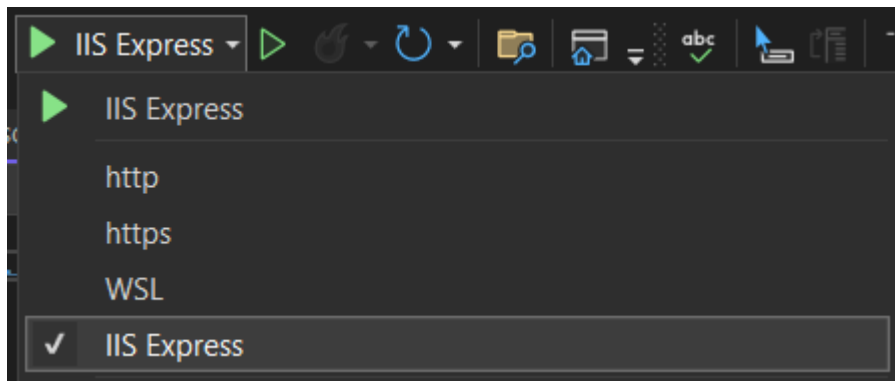
IIS (Internet Information Services) — веб-сервер от Microsoft, входящий в состав Windows Server. Традиционно используется для развёртывания ASP.NET-приложений на Windows-серверах. Поддерживает глубокую интеграцию с экосистемой Microsoft.

Apache — один из старейших и наиболее распространённых веб-серверов. Используется реже Nginx для задач обратного прокси, однако является полноценной альтернативой.

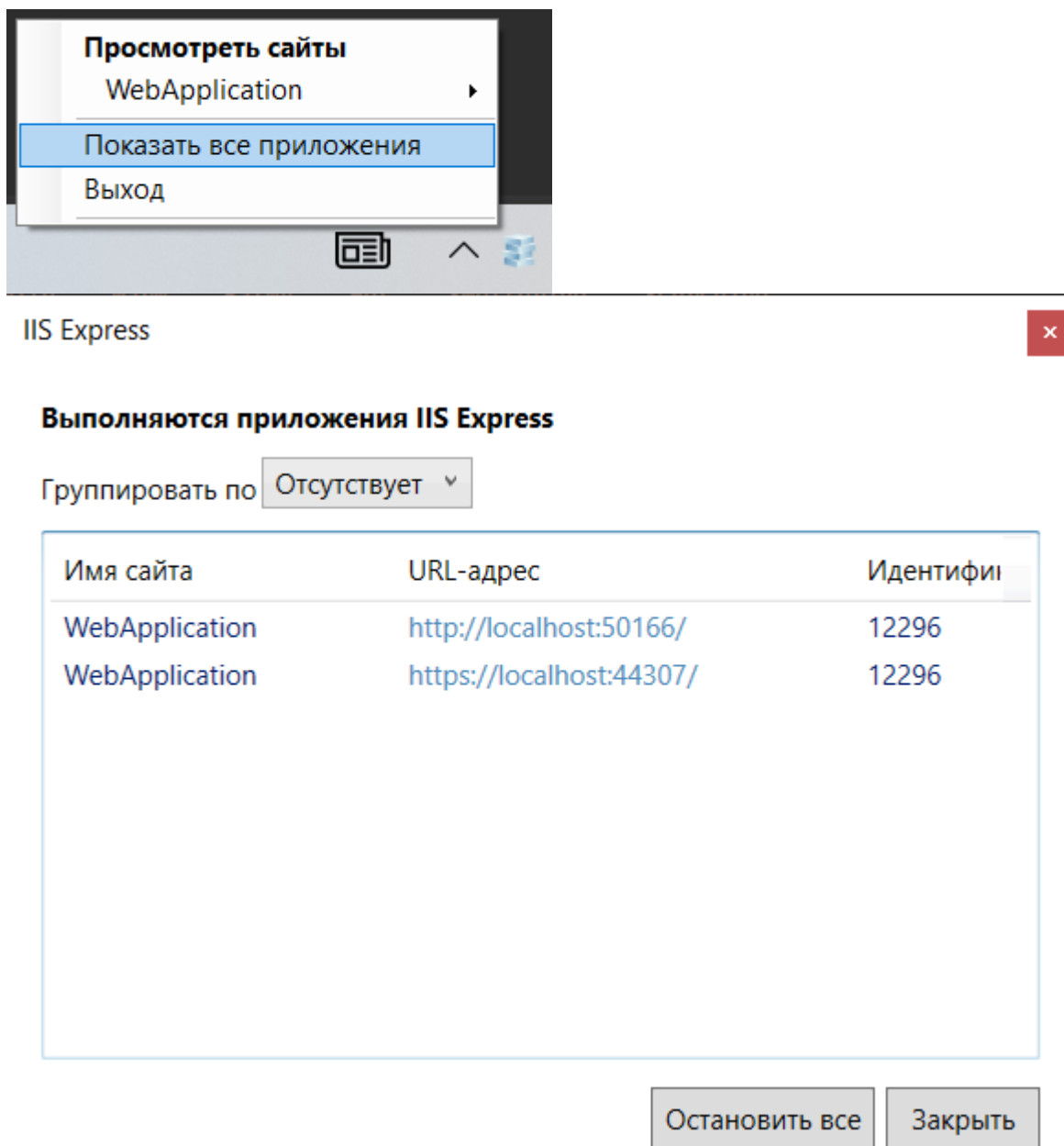
IIS Express

Прежде чем переходить к настройке Nginx на удалённом сервере, полезно рассмотреть принцип работы обратного прокси на простом примере. **IIS Express** — это облегчённая версия IIS, которая устанавливается вместе с Visual Studio и используется для локальной разработки.

При запуске ASP.NET-проекта из Visual Studio можно выбрать профиль запуска через IIS Express в выпадающем списке рядом с кнопкой запуска:



После запуска IIS Express появляется в системном трее. Кликнув по его иконке и выбрав «Показать все приложения», можно увидеть, по каким адресам он принимает запросы:



Можно заметить, что приложение теперь доступно по портам 50166 (HTTP) и 44307 (HTTPS), хотя в коде приложения порт 44307 и поддержка HTTPS не настраивались. Это объясняется именно наличием прокси: IIS Express принимает запросы на своих портах и перенаправляет их к Kestrel, который слушает внутренний порт. Разработчику не нужно ничего настраивать вручную — достаточно выбрать профиль и нажать кнопку запуска.

Эти параметры описываются в файле `launchSettings.json`. В нём присутствуют два профиля: профиль `http` запускает приложение напрямую через Kestrel на порту 5004, профиль `IIS Express` запускает приложение через IIS Express, который слушает порты 50166 и 44307 и проксирует запросы к Kestrel:

```
{
  "profiles": {
    "http": {
      "commandName": "Project",
      "environmentVariables": {
```

```
    "ASPNETCORE_ENVIRONMENT": "Development"
  },
  "dotnetRunMessages": true,
  "applicationUrl": "http://localhost:5004"
},
"IIS Express": {
  "commandName": "IISExpress",
  "launchBrowser": true,
  "environmentVariables": {
    "ASPNETCORE_ENVIRONMENT": "Development"
  }
}
},
"iisSettings": {
  "windowsAuthentication": false,
  "anonymousAuthentication": true,
  "iisExpress": {
    "applicationUrl": "http://localhost:50166/",
    "sslPort": 44307
  }
}
}
```

Параметры IIS Express вынесены в отдельную секцию `iisSettings`: там указан порт HTTP 50166 и порт HTTPS 44307. Kestrel при этом продолжает слушать свой внутренний порт 5004 — клиенты его не видят и обращаются исключительно к IIS Express.

IIS Express удобен для разработки, однако не предназначен для эксплуатации: он не рассчитан на высокую нагрузку, не работает как системная служба и не запускается автоматически при старте сервера. В продакшене на Windows используется полноценный IIS, а на Linux — Nginx или Apache.

3. Развёртывание на удалённом Linux-сервере вручную

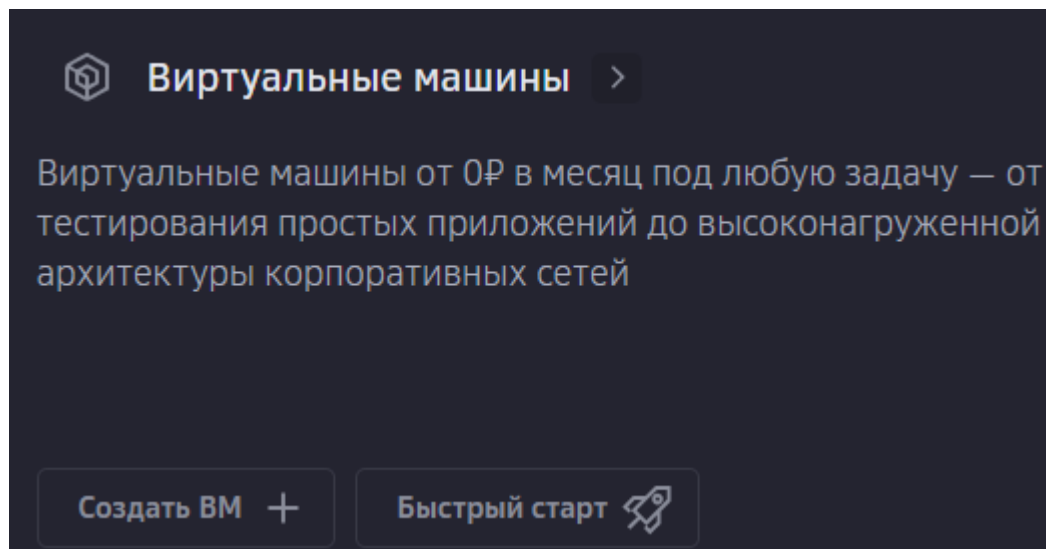
Подготовка сервера

Для развёртывания ASP.NET Web API необходим удалённый сервер с доступом по SSH. Существует несколько вариантов размещения.

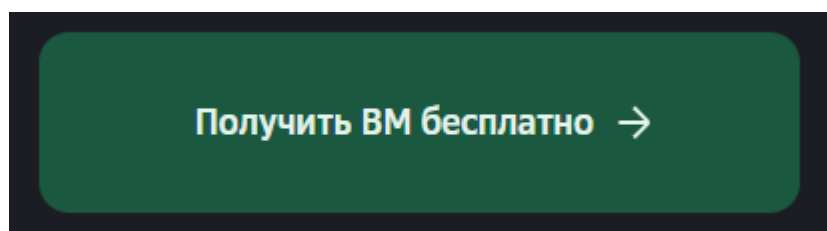
VPS-хостинг (Virtual Private Server) — аренда виртуального выделенного сервера у хостинг-провайдера. VPS представляет собой изолированную виртуальную машину, которая работает на физическом сервере провайдера. Каждый VPS имеет выделенные ресурсы — процессор, память и дисковое пространство — и полностью независим от других виртуальных машин на том же физическом сервере. VPS предоставляет полный контроль над операционной системой при относительно невысокой стоимости.

Облачные платформы — Azure, AWS, Google Cloud. Предоставляют широкий спектр дополнительных сервисов: управляемые базы данных, хранилища файлов, балансировщики нагрузки и многое другое. Подходят для более зрелых проектов с требованиями к масштабируемости.

В рамках данной лекции мы будем работать с хостинг-провайдером Cloud.ru, при работе с другими провайдерами процесс будет аналогичным. После регистрации и создания личного облака можно приступить к созданию виртуальной машины.



В данном сервисе есть тариф для создания бесплатной VPS, выберем этот вариант.



Рассматривается VPS с операционной системой Ubuntu 22.04 LTS. Для учебного проекта достаточно минимальной конфигурации: 2 vCPU, 4 ГБ RAM, 30 ГБ дискового пространства. Так как мы будем публиковать Web API приложение, нам понадобится публичный IP-адрес, по которому клиенты смогут отправлять запросы на наш сервер. Публичный IP выходит за рамки бесплатного тарифа, поэтому за него нужно будет заплатить небольшую сумму.

Создание виртуальной машины (free tier)

Бесплатная виртуальная машина

Название

vm-78асеа

9/64

Образ

Публичные

Маркетплейс

Ubuntu 22.04

Вычислительный ресурс

vCPU

2 шт.

RAM

4 ГБ

Гарантированная доля vCPU

10%

Диск

30 ГБ

Подключить публичный IP

Публичный IP

Арендовать новый

Авторизация пользователя

Логин

user1

Имя пользователя для входа в ВМ

Метод аутентификации

Пароль

Публичный ключ

Пароль

.....

Для доступа к ВМ через виртуальную консоль

⚠

Рекомендуем аутентификацию по SSH-ключу. Этот способ исключает риск, связанный с подбором и взломом вашего пароля.

Дополнительные параметры системы

Отмена

Создать

Итого в месяц с учетом НДС

149,33 ₽

Стоимость зависит от потребления

Детализация заказа

Публичный IP

149,33 ₽

Стоимость: 149,33 ₽

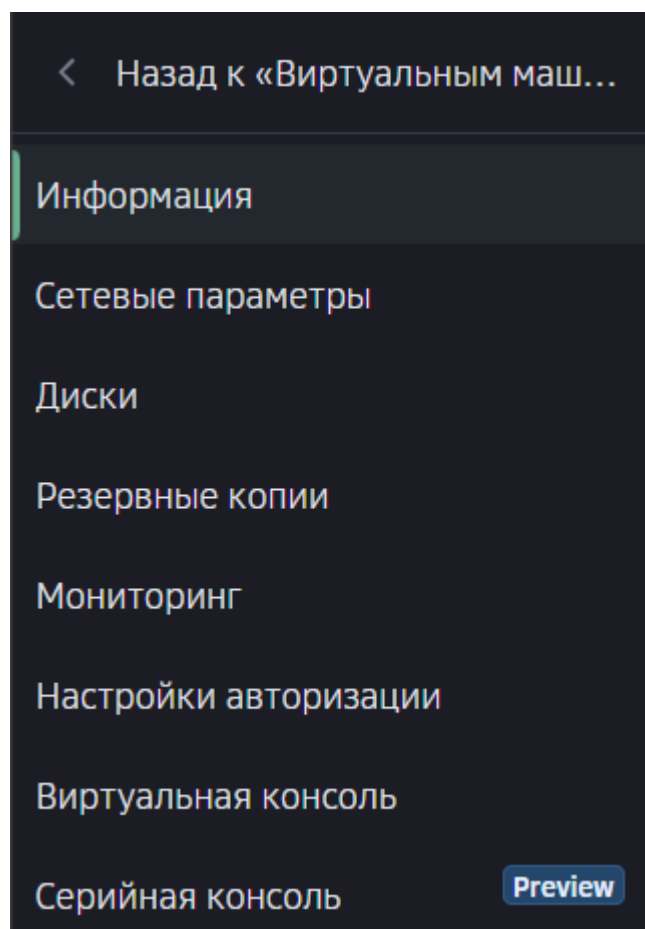
Подробнее о тарифах и ценах

Документация

5

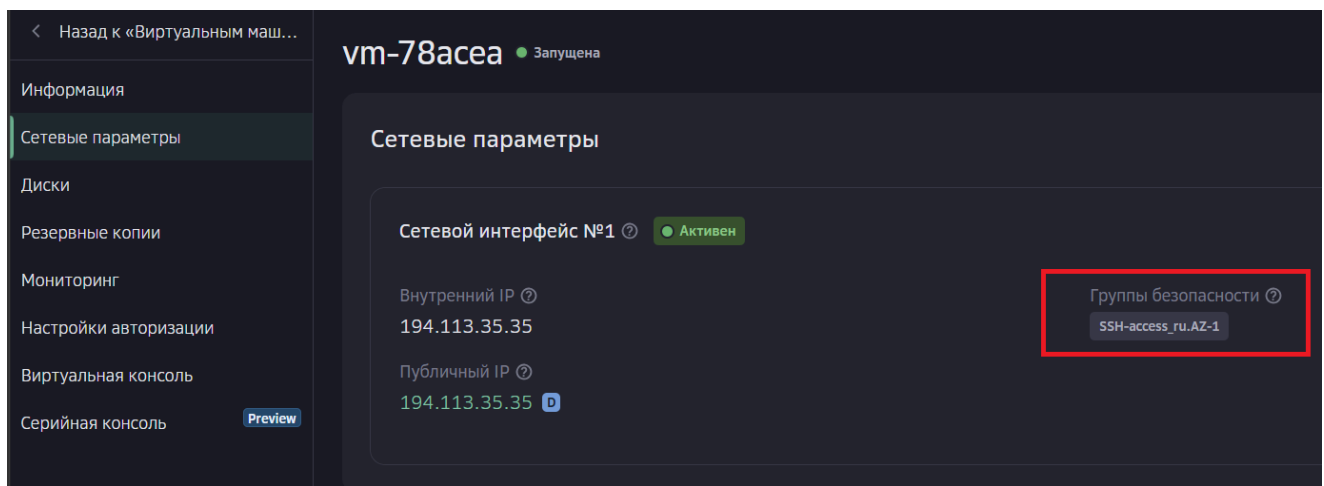
После создания сервера провайдер предоставляет публичный IP-адрес и данные для подключения.

Виртуальные машины					
☐		Поиск			
Зона доступности: Все ×		Статус: Все ×	+ Добавить	× Очистить	
	Название	Зона доступности	Статус	Внутренний IP	Публичный IP
☐	vm-78acea Free tier	ru.AZ-1	Запущена	—	194.113.35.35

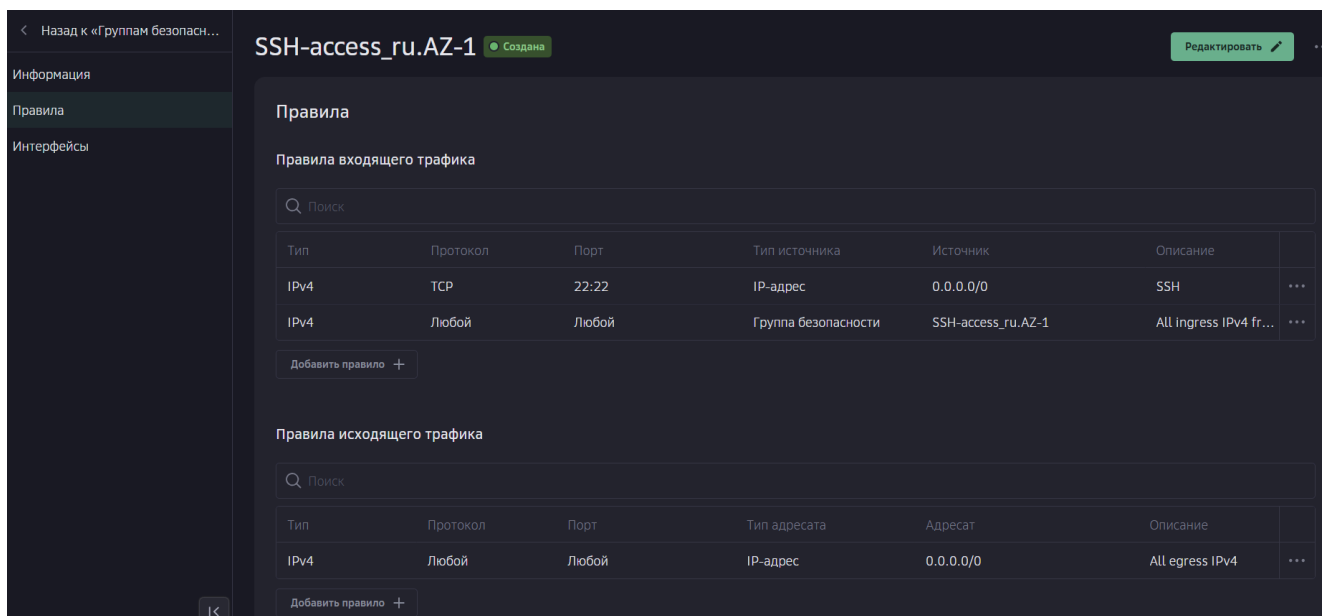


Подключение к серверу выполняется по протоколу **SSH (Secure Shell)** — криптографическому сетевому протоколу, который обеспечивает защищённое удалённое управление сервером через командную строку. Весь трафик между клиентом и сервером шифруется, что исключает перехват команд и передаваемых данных.

Для того чтобы можно было делать запросы к VPS, нужно отредактировать группу безопасности:



По умолчанию извне доступны только SSH-запросы. Нам нужно добавить правила для портов 80 (HTTP) и 443 (HTTPS). В поле источника указывается значение `0.0.0.0/0` — это означает, что запросы разрешены с любого IP-адреса. Такая запись является стандартным способом обозначения «весь интернет» в правилах сетевого доступа:



Создание правила



Протокол *

TCP



Порт

80



Тип источника *

IP-адрес



Источник *

0.0.0.0/0



Введите IP-адрес с префиксом

Описание

HTTP



4/255

Отмена

Создать

Перед входом через SSH с паролем настроим VPS. Для этого:

- **Зайдите в консоль сервера** через панель управления провайдера.

```
< Назад к «Виртуальным маш... Ctrl Alt Win Tab Esc

Информация
Сетевые параметры
Диски
Резервные копии
Мониторинг
Настройки авторизации
Виртуальная консоль
Серийная консоль Preview

Ubuntu 22.04.5 LTS vm-78acea tty1
vm-78acea login: user1
Password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-161-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro

System information as of Sun May 10 03:59:41 PM MSK 2026

System load:  0.12           Processes:    137
Usage of /:   16.5% of 29.42GB Users logged in:  0
Memory usage: 11%           IPv4 address for enp3s0: 194.113.35.35
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

*** System restart required ***

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

$
```

- **Откройте файл конфигурации SSH:** `sudo nano /etc/ssh/sshd_config`
- **Найдите и измените следующие строки:**
 - `PasswordAuthentication yes`
 - `KbdInteractiveAuthentication yes`

```
GNU nano 6.2 /etc/ssh/sshd_config *
#LoginGraceTime 2m
#PermitRootLogin prohibit-password
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2

#AuthorizedPrincipalsFile none

#AuthorizedKeysCommand none
#AuthorizedKeysCommandUser nobody

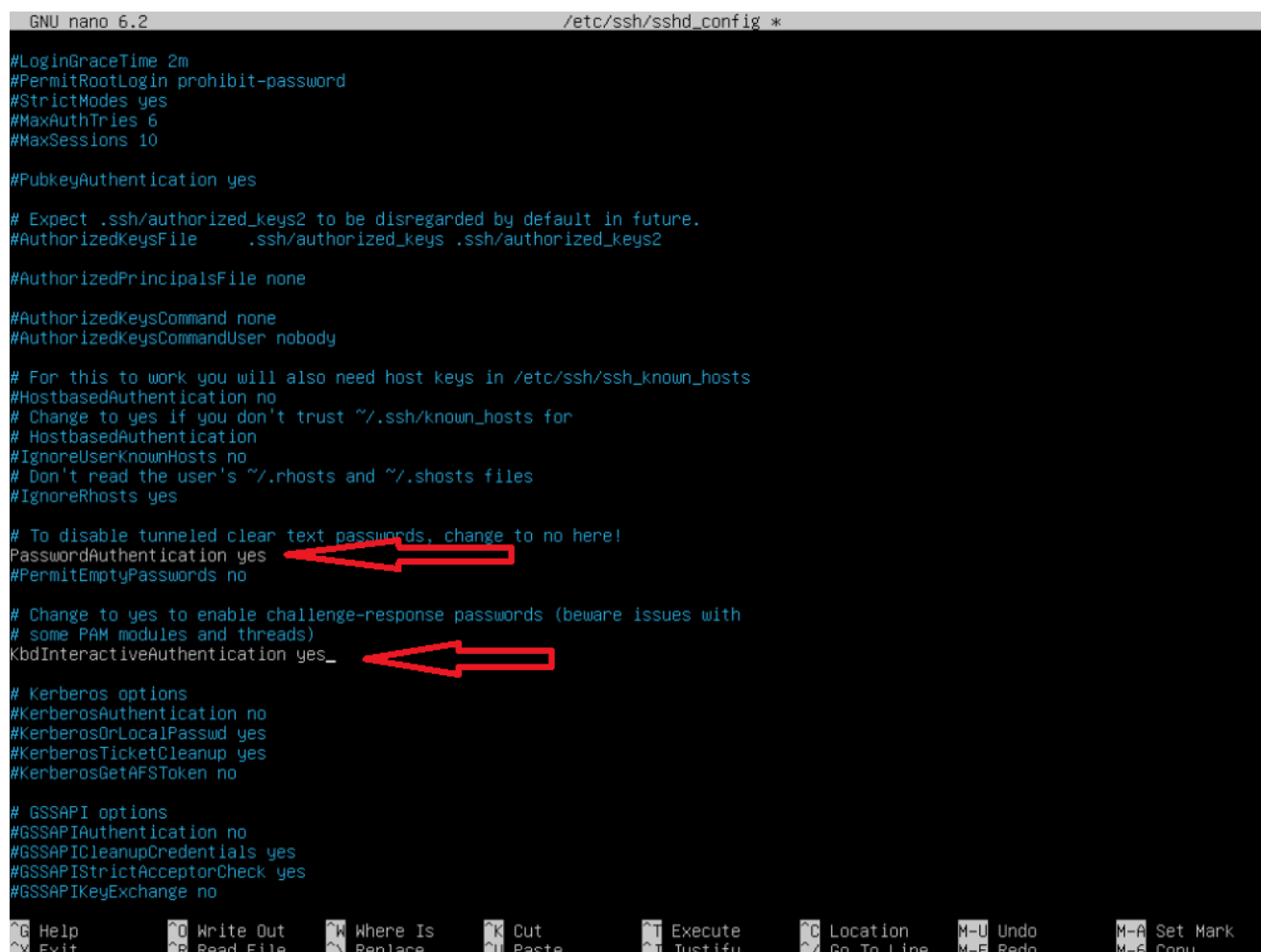
# For this to work you will also need host keys in /etc/ssh/ssh_known_hosts
#HostbasedAuthentication no
# Change to yes if you don't trust ~/.ssh/known_hosts for
# HostbasedAuthentication
#IgnoreUserKnownHosts no
# Don't read the user's ~/.rhosts and ~/.shosts files
#IgnoreRhosts yes

# To disable tunneled clear text passwords, change to no here!
PasswordAuthentication yes
#PermitEmptyPasswords no

# Change to yes to enable challenge-response passwords (beware issues with
# some PAM modules and threads)
KbdInteractiveAuthentication yes_

# Kerberos options
#KerberosAuthentication no
#KerberosOrLocalPasswd yes
#KerberosTicketCleanup yes
#KerberosGetAFSToken no

# GSSAPI options
#GSSAPIAuthentication no
#GSSAPICleanupCredentials yes
#GSSAPIStrictAcceptorCheck yes
#GSSAPIKeyExchange no
```



- **Сохраните изменения:** нажмите `Ctrl+O` , затем `Enter` , и выйдите через `Ctrl+X` .
- **Перезапустите службу SSH:** `sudo systemctl restart sshd`

После этого можно подключиться к удалённому серверу через SSH с паролем со своего компьютера. Введите в терминале:

```
Windows PowerShell
PS C:\Users\andro> ssh user1@194.113.35.35
(user1@194.113.35.35) Password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-161-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun May 10 04:20:23 PM MSK 2026

System load:  0.07               Processes:            140
Usage of /:   16.6% of 29.42GB   Users logged in:     1
Memory usage: 11%               IPv4 address for enp3s0: 194.113.35.35
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

*** System restart required ***
Last login: Sun May 10 15:59:41 2026
$ |
```

При первом подключении система запросит подтверждение отпечатка ключа сервера. После ввода `yes` и пароля выполняется вход в командную строку удалённого сервера. Для повышения безопасности рекомендуется настроить аутентификацию по SSH-ключу вместо пароля, однако детальное рассмотрение этой темы выходит за рамки данной лекции.

После подключения перейдите под права суперпользователя:

```
sudo -i
```

Рекомендуется обновить пакеты операционной системы:

```
apt update && apt upgrade -y
```

Установка окружения

На сервере необходимо установить среду выполнения .NET. Поскольку приложение будет публиковаться в режиме framework-dependent, достаточно установить только Runtime без SDK.

Для установки .NET на Ubuntu необходимо сначала подключить официальный репозиторий Microsoft:

```
# Загрузка и регистрация пакета репозитория Microsoft
wget https://packages.microsoft.com/config/ubuntu/22.04/packages-microsoft-prod.deb
dpkg -i packages-microsoft-prod.deb
rm packages-microsoft-prod.deb

# Обновление списка пакетов и установка .NET Runtime
# Выберите версию, соответствующую версии вашего проекта
apt update
apt install -y dotnet-runtime-9.0
apt install -y aspnetcore-runtime-9.0
```

Проверка корректности установки:

```
dotnet --info
```

```
root@vm-78acea:~# dotnet --info
Host:
  Version:      9.0.4
  Architecture: x64
  Commit:       f57e6dc747
  RID:          linux-x64

.NET SDKs installed:
  No SDKs were found.

.NET runtimes installed:
  Microsoft.NETCore.App 9.0.4 [/usr/share/dotnet/shared/Microsoft.NETCore.App]

Other architectures found:
  None

Environment variables:
  Not set

global.json file:
  Not found

Learn more:
  https://aka.ms/dotnet/info

Download .NET:
  https://aka.ms/dotnet/download
root@vm-78acea:~#
```

Копирование файлов на сервер

На локальной машине выполняется сборка публикации:

```
dotnet publish -c Release -o ./publish
```

Копирование папки `publish` на удалённый сервер выполняется через утилиту `scp` (Secure Copy Protocol). Команда выполняется на локальной машине:

```
scp -r ./publish user1@YOUR_SERVER_IP:~/myapp
```

Флаг `-r` означает рекурсивное копирование директории. Путь `~/myapp` — это целевая директория на сервере в домашней директории пользователя, полный путь будет `/home/user1/myapp`.

В Linux принято размещать веб-приложения в директории `/var/www`. Создадим её на сервере (требуется права `root`):

```
mkdir -p /var/www/myapp
```

Если на сервере уже есть старая версия приложения, удалим её:

```
rm -rf /var/www/myapp/*
```

Переместим скопированные файлы из домашней директории пользователя в `/var/www/myapp/`:

```
mv /home/user1/myapp/* /var/www/myapp/
```

После копирования следует убедиться, что приложение запускается корректно, выполнив его вручную на сервере:

```
cd /var/www/myapp  
ls  
dotnet WebApplication.dll
```

Если приложение запустилось без ошибок и Kestrel начал прослушивать порт, можно переходить к настройке Nginx. Для остановки приложения используется сочетание клавиш `Ctrl+C` — в дальнейшем запуском будет управлять `systemd`.

```
root@vm-78acea:/var/www/myapp# dotnet WebApplication.dll
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Production
info: Microsoft.Hosting.Lifetime[0]
      Content root path: /var/www/myapp
```

Настройка Nginx

Установка Nginx:

```
apt install -y nginx
```

После установки Nginx автоматически запускается и начинает прослушивать порт 80. Для проверки можно открыть в браузере `http://YOUR_SERVER_IP` — должна отобразиться стандартная страница приветствия Nginx.

Конфигурация Nginx хранится в директории `/etc/nginx`. Для каждого приложения создаётся отдельный файл конфигурации в директории `/etc/nginx/sites-available`, после чего на него создаётся символическая ссылка в `/etc/nginx/sites-enabled`.

Создание файла конфигурации:

```
nano /etc/nginx/sites-available/myapp
```

Содержимое файла конфигурации:

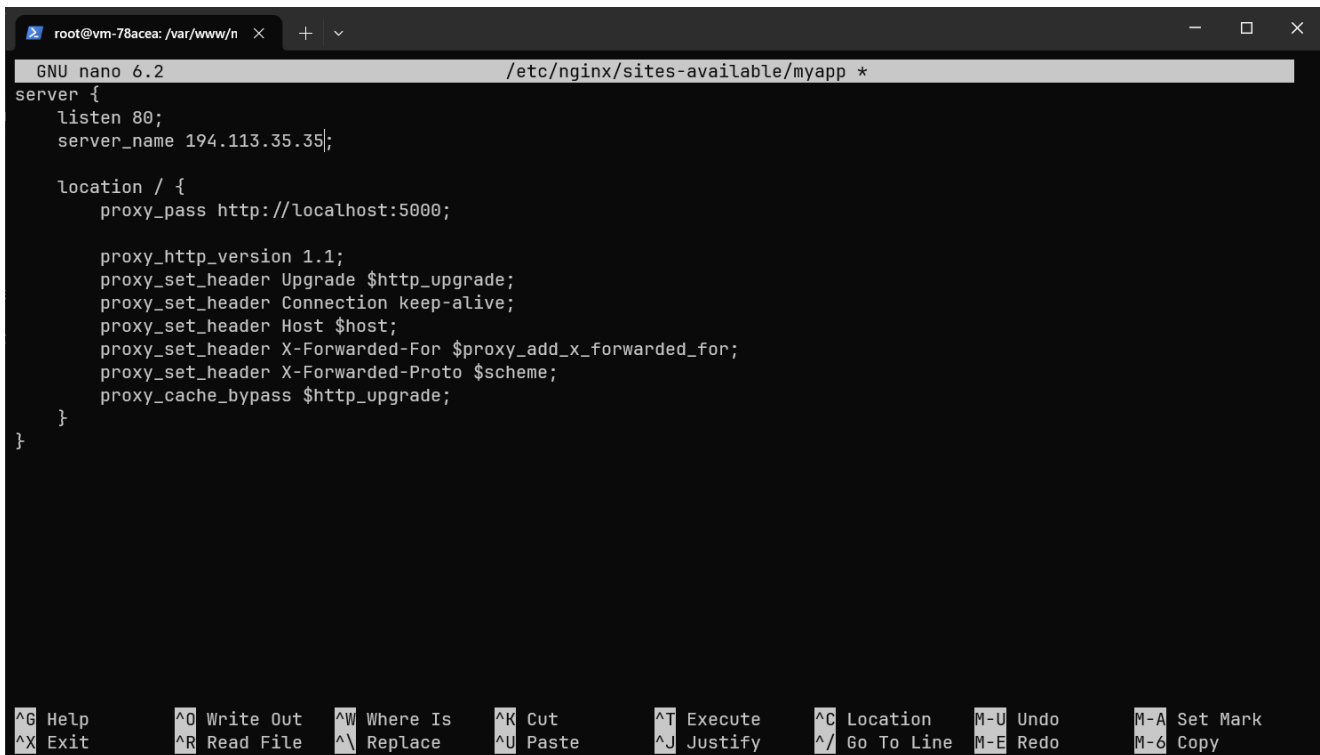
```
server {
    listen 80;
    server_name YOUR_SERVER_IP;

    location / {
        proxy_pass http://localhost:5000;

        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection keep-alive;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }
}
```

```
}  
}
```

Директива `proxy_pass` указывает адрес, на который Nginx перенаправляет входящие запросы — в данном случае это локальный порт Kestrel. Заголовки `X-Forwarded-For` и `X-Forwarded-Proto` передают приложению информацию о реальном IP-адресе клиента и протоколе исходного запроса. Без этих заголовков приложение будет видеть только адрес самого Nginx вместо адреса клиента.



```
GNU nano 6.2 /etc/nginx/sites-available/myapp *  
server {  
    listen 80;  
    server_name 194.113.35.35;  
  
    location / {  
        proxy_pass http://localhost:5000;  
  
        proxy_http_version 1.1;  
        proxy_set_header Upgrade $http_upgrade;  
        proxy_set_header Connection keep-alive;  
        proxy_set_header Host $host;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
        proxy_cache_bypass $http_upgrade;  
    }  
}
```

Чтобы ASP.NET корректно обрабатывал эти заголовки, необходимо подключить соответствующий middleware в `Program.cs`. Если это не настроить, приложение будет получать IP-адрес и протокол прокси-сервера, а не настоящего клиента:

```
// Program.cs  
app.UseForwardedHeaders(new ForwardedHeadersOptions  
{  
    ForwardedHeaders = ForwardedHeaders.XForwardedFor |  
                       ForwardedHeaders.XForwardedProto  
});
```

Этот middleware должен располагаться в самом начале конвейера — до `UseAuthentication`, `UseAuthorization` и других middleware.

После создания файла конфигурации необходимо активировать его, создав символическую ссылку:

```
ln -s /etc/nginx/sites-available/myapp /etc/nginx/sites-enabled/myapp
```

Проверка корректности конфигурации и перезапуск Nginx:

```
nginx -t  
systemctl restart nginx
```

Команда `nginx -t` проверяет синтаксис конфигурационных файлов и сообщает об ошибках до перезапуска сервиса. Рекомендуется всегда выполнять эту проверку перед перезапуском.

Настройка автозапуска через systemd

В текущем состоянии приложение необходимо запускать вручную командой `dotnet WebApplication.dll`. При перезагрузке сервера или аварийном завершении процесса приложение останавливается и не восстанавливается автоматически. Для решения этой проблемы используется **systemd** — системный менеджер служб Linux, отвечающий за запуск и управление фоновыми процессами.

Создание unit-файла службы:

```
nano /etc/systemd/system/myapp.service
```

Содержимое unit-файла:

```
[Unit]  
Description=MyApp ASP.NET Web API  
After=network.target  
  
[Service]  
WorkingDirectory=/var/www/myapp  
ExecStart=/usr/bin/dotnet /var/www/myapp/WebApplication.dll  
Restart=always  
RestartSec=10  
KillSignal=SIGINT  
User=www-data  
Environment=ASPNETCORE_ENVIRONMENT=Production  
Environment=ASPNETCORE_URLS=http://localhost:5000  
  
[Install]  
WantedBy=multi-user.target
```

Директива `After=network.target` гарантирует, что служба запустится только после инициализации сетевых интерфейсов. Директива `Restart=always` обеспечивает автоматический перезапуск приложения при его аварийном завершении. Параметр `RestartSec=10` задаёт задержку в 10 секунд перед повторным запуском. Директива `User=www-data` указывает, от имени какого системного пользователя будет запущен процесс — запуск от имени `root` не рекомендуется из соображений безопасности.

Секреты и параметры конфигурации передаются через директивы `Environment` в unit-файле. Как было рассмотрено в лекции по безопасности, это предпочтительный способ передачи секретов в продакшен-окружении:

```
[Service]
Environment=ASPNETCORE_ENVIRONMENT=Production
Environment=ASPNETCORE_URLS=http://localhost:5000
Environment=JwtSettings__SecretKey=my-production-secret-key
Environment=ConnectionStrings__DefaultConnection=Data
Source=/var/www/myapp/app.db
```

После создания unit-файла необходимо перезагрузить конфигурацию `systemd`, запустить службу и добавить её в автозагрузку:

```
# Перезагрузка конфигурации systemd
systemctl daemon-reload

# Запуск службы
systemctl start myapp

# Добавление в автозагрузку
systemctl enable myapp

# Проверка статуса
systemctl status myapp
```

Вывод команды `systemctl status myapp` показывает текущее состояние службы, последние строки лога и информацию о процессе. Если служба запущена корректно, в выводе будет присутствовать строка `Active: active (running)`.

Проверка работы

После настройки Nginx и systemd API должен быть доступен по публичному IP-адресу сервера. Проверка выполняется обращением к любому endpoint приложения:

```
curl http://YOUR_SERVER_IP/api/products
```

Если ответ получен корректно, развёртывание выполнено успешно.

```

PS C:\Users\andro\Desktop\test\WebApplication> curl http://194.113.35.35/weatherforecast

StatusCode      : 200
StatusDescription : OK
Content         : [{"date":"2026-05-11","temperatureC":8,"summary":"Balmy", "temperatureF":46}, {"date":"2026-05-12","temperatureC":50,"summary":"Scorching", "temperatureF":121}, {"date":"2026-05-13","temperatureC":32,"sum...
RawContent      : HTTP/1.1 200 OK
                  Transfer-Encoding: chunked
                  Connection: keep-alive
                  Content-Type: application/json; charset=utf-8
                  Date: Sun, 10 May 2026 17:12:11 GMT
                  Server: nginx/1.18.0 (Ubuntu)

                  [{"date":"2026...
Forms           : {}
Headers         : {[Transfer-Encoding, chunked], [Connection, keep-alive], [Content-Type, application/json; charset=utf-8], [Date, Sun, 10 May 2026 17:12:11 GMT]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 391

```

При возникновении проблем первым инструментом диагностики является просмотр логов приложения через `journalctl`:

```

# Просмотр последних логов службы
journalctl -u myapp -n 50

# Просмотр логов в режиме реального времени
journalctl -u myapp -f

```

Флаг `-u myapp` фильтрует логи по имени службы. Флаг `-n 50` выводит последние 50 строк. Флаг `-f` включает режим реального времени — новые строки лога выводятся по мере их появления.

Помимо логов приложения, полезно проверить логи Nginx, которые хранятся в `/var/log/nginx/`:

```

# Лог доступа — все входящие запросы
tail -f /var/log/nginx/access.log

# Лог ошибок
tail -f /var/log/nginx/error.log

```

4. Развёртывание с использованием Docker

Проблема воспроизводимости окружения

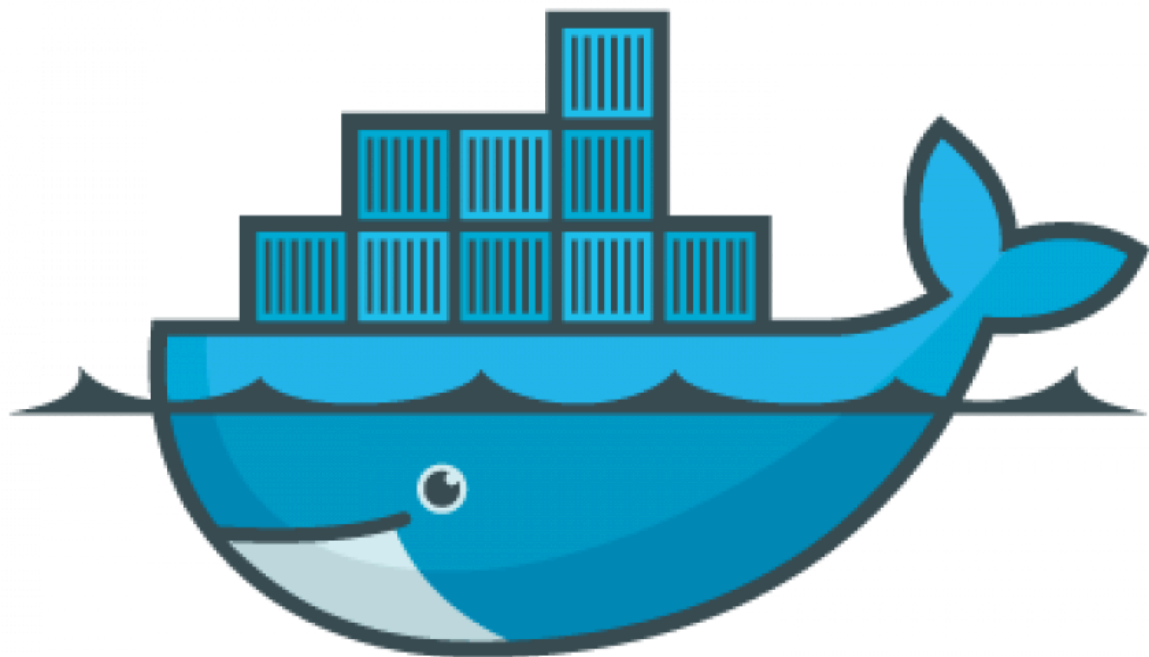
При ручном развёртывании, рассмотренном в предыдущем разделе, на сервере вручную устанавливается .NET Runtime, настраивается Nginx и создаётся unit-файл systemd. Такой подход работает, однако имеет существенный недостаток: окружение сервера формируется вручную и нигде не описано в виде кода. Если потребуется развернуть второй сервер, перенести приложение на другой хостинг или воспроизвести проблему локально — все шаги придётся повторять заново и по памяти.

Ещё более распространённая проблема: приложение корректно работает на машине разработчика, но ведёт себя иначе на сервере. Причиной могут быть различия в версии .NET, параметрах операционной системы или установленных зависимостях.

Для решения этих проблем и обеспечения воспроизводимого и изолированного окружения для запуска приложения используется Docker.

Контейнеры и образы

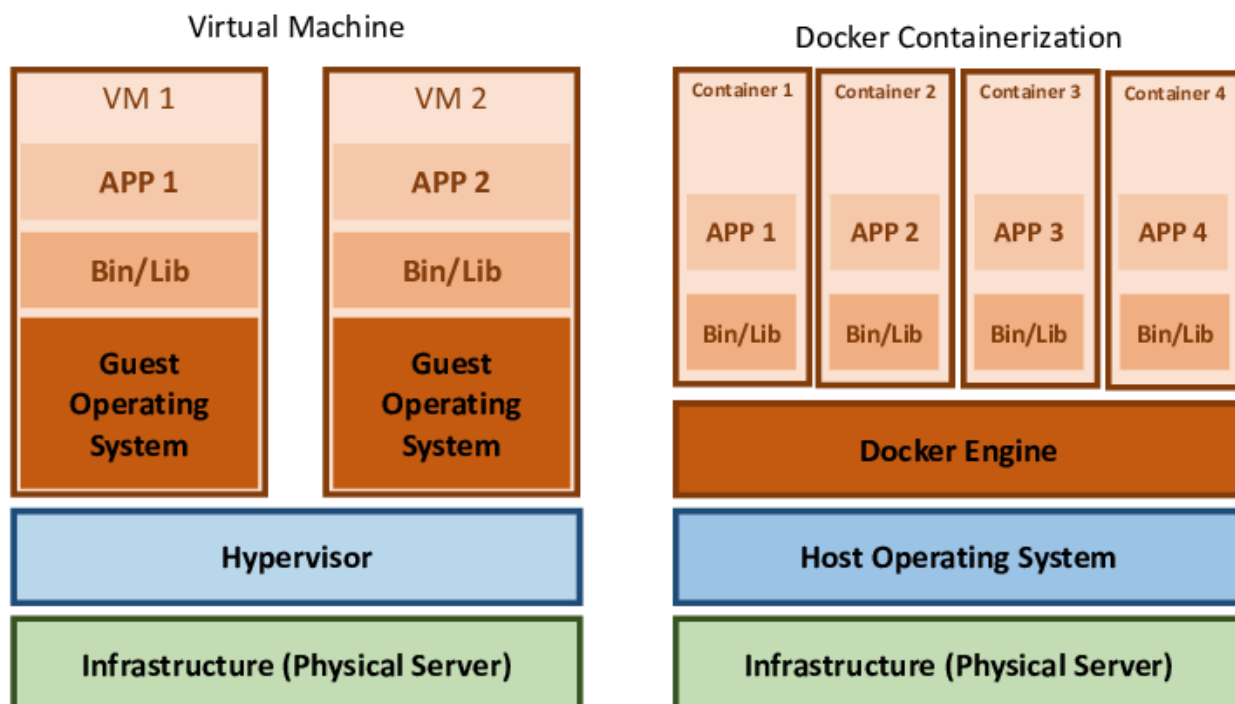
Docker — это платформа для запуска приложений в изолированных окружениях, называемых контейнерами. Чтобы понять как это работает, полезно разобраться в нескольких ключевых понятиях.



docker

Контейнер — это изолированный процесс, который запускается со своей собственной файловой системой, сетью и набором зависимостей. Контейнер не знает ничего об операционной системе хоста и о других контейнерах, запущенных рядом с ним. При этом контейнер не является полноценной виртуальной машиной — он использует ядро операционной системы хоста и поэтому запускается значительно быстрее и потребляет значительно меньше ресурсов.

Разница между контейнером и виртуальной машиной наглядно демонстрируется следующей схемой:



Виртуальная машина включает полноценную гостевую операционную систему, что делает её тяжёлой и медленной в запуске. Контейнер включает только само приложение и его зависимости — без операционной системы.

Образ (Image) — это неизменяемый шаблон, на основе которого создаётся контейнер. Образ содержит всё необходимое для запуска приложения: операционное окружение, среду выполнения, зависимости и сам код приложения. Отношение образа к контейнеру можно сравнить с отношением класса к объекту: образ — это описание, контейнер — это запущенный экземпляр.

Docker Hub — публичный реестр образов, где размещаются готовые образы для большинства популярных технологий. Именно оттуда Docker скачивает базовые образы, например официальный образ .NET Runtime от Microsoft. [Docker Hub Container Image Library](#) | [App Containerization](#)

Dockerfile

Инструкция по сборке образа описывается в специальном файле — `Dockerfile`. Это текстовый файл без расширения, который размещается в корне проекта и содержит последовательность команд для формирования образа.

Для ASP.NET Web API типичный `Dockerfile` использует многоэтапную сборку (multi-stage build). Это означает, что сборка выполняется в два этапа: на первом этапе используется образ с SDK для компиляции и публикации приложения, на втором — лёгкий образ только с Runtime для запуска. Такой подход позволяет получить минимальный по размеру итоговый образ, не включающий инструменты сборки:

```
# Этап 1: сборка и публикация
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
```

```
WORKDIR /src

# Копирование файла проекта и восстановление зависимостей
COPY MyApp.csproj .
RUN dotnet restore

# Копирование остального кода и публикация
COPY . .
RUN dotnet publish -c Release -o /app/publish

# Этап 2: финальный образ для запуска
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
WORKDIR /app

# Копирование результатов публикации из первого этапа
COPY --from=build /app/publish .

EXPOSE 8080
ENTRYPOINT ["dotnet", "MyApp.dll"]
```

Директива `FROM` указывает базовый образ. На первом этапе используется образ `dotnet/sdk`, содержащий полный SDK для сборки. На втором этапе используется образ `dotnet/aspnet` — значительно меньший образ, содержащий только ASP.NET Runtime. Директива `EXPOSE` объявляет порт, который приложение будет прослушивать внутри контейнера. Директива `ENTRYPOINT` определяет команду, которая выполняется при запуске контейнера.

Как Docker меняет процесс развёртывания

С использованием Docker последовательность развёртывания существенно упрощается по сравнению с ручным подходом. Рассмотрим как меняется каждый шаг.

Локально разработчик собирает образ из `Dockerfile` и запускает контейнер. Приложение работает в том же окружении, что и на сервере, что исключает расхождения между средами.

На сервере вместо установки .NET Runtime устанавливается только Docker Engine. Это единственная зависимость, которая требуется на сервере — всё остальное окружение описано внутри образа:

```
# Установка Docker на Ubuntu
apt install -y docker.io
systemctl enable docker
```

Образ доставляется на сервер одним из двух способов. Первый — публикация образа в реестр (Docker Hub или приватный реестр) и скачивание его на сервере.

Второй — сборка образа непосредственно на сервере путём копирования исходного кода или `Dockerfile`.

Запуск приложения выполняется одной командой вместо настройки `systemd`:

```
docker run -d \
  --name myapp \
  --restart always \
  -p 5000:8080 \
  -e ASPNETCORE_ENVIRONMENT=Production \
  -e JwtSettings__SecretKey=my-production-secret-key \
  -e ConnectionStrings__DefaultConnection="Data Source=/app/app.db" \
  myapp:latest
```

Флаг `-d` запускает контейнер в фоновом режиме. Флаг `--restart always` обеспечивает автоматический перезапуск контейнера при его остановке или перезагрузке сервера — аналог `Restart=always` в `systemd`. Флаг `-p 5000:8080` пробрасывает порт: запросы на порт `5000` хоста будут перенаправляться на порт `8080` внутри контейнера. Флаги `-e` передают переменные окружения в контейнер.

Nginx остаётся в схеме развёртывания и настраивается точно так же, как и при ручном подходе — конфигурация не меняется. Nginx по-прежнему принимает входящие запросы и перенаправляет их на локальный порт, который теперь слушает контейнер, а не напрямую Kestrel.

Обновление приложения сводится к остановке старого контейнера и запуску нового с обновлённым образом:

```
docker stop myapp
docker rm myapp
docker run -d --name myapp ...
```

Преимущества подхода

Использование Docker даёт несколько практических преимуществ по сравнению с ручным развёртыванием.

Воспроизводимость. Образ содержит полное описание окружения. Приложение будет вести себя одинаково на машине разработчика, на тестовом сервере и в продакшене. Фраза «у меня работает» теряет актуальность: если образ собирается и запускается локально, он будет работать и на сервере.

Изоляция. Несколько приложений могут работать на одном сервере в отдельных контейнерах, не влияя друг на друга. Каждый контейнер имеет собственные зависимости и может использовать разные версии .NET одновременно.

Простота обновления. Обновление приложения сводится к замене контейнера. Не нужно вручную копировать файлы, перезапускать службы и следить за тем, чтобы старые файлы были корректно заменены новыми.

Переносимость. Образ можно запустить на любом сервере с установленным Docker, независимо от хостинг-провайдера и конфигурации операционной системы.